> **NOTE** Starting with version 7.x, any index created is set to be backed up by a single shard and a replica by default. The index's shards and replicas can be configured and customized based on data size needs. While replicas can be resized, the number of shards cannot be modified on a live index.

Shards can be categorized as primary and replicas. *Primary* shards hold documents, whereas *replica* shards (or simply *replicas*), as the name suggests, are copies of primary shards. We can have one or more replicas for each shard. We can also have no replicas, but such a design is not advisable for production environments—in real-world production environments, we create multiple replicas for each shard. Replicas hold data copies, increasing the system's redundancy and thus helping speed up search queries. We learn more about shards and replicas in the next section.

An index can hold any number of documents (but as we discussed earlier, only one document type), so it is advisable to find an optimum size for your needs (we discuss shard sizing in chapter 14). Each index can either exist on a single node or be distributed across multiple nodes in the cluster. Using the design in figure 3.6 (one index with one shard and two replicas), every document we index will be stored as three copies: one in a shard and two copies on the replicas.

> **NOTE** Optimal index sizing must be carried out before setting up the index. The size depends on the data we may have to serve today as well as the estimated size for the future. Replicas create additional copies of data, so be sure you create capacity considering this. We look at index sizing in chapter 14.

Every index has properties like mappings, settings, and aliases. Mapping is a process of defining schema definitions, while settings let us configure the shards and replicas. Aliases are alternate names given to a single index or a set of indexes. A few settings, such as the number of replicas, can be set dynamically; but other properties, like the number of shards, cannot be changed when the index is in operation. We should ideally create templates for the configuration of any new indexes.

We use RESTful APIs to work on indexes: creating and deleting indexes, changing their settings, closing and opening them, reindexing data, and other operations. We work through these APIs extensively in chapter 6.

3.2.3 Data streams

Indexes (such as `movies`, `movie_reviews`, etc.) hold and collect data. Over time, they can become huge as more data is accumulated and stored. Adding nodes can alleviate this problem by distributing shards across more nodes. The expectation is that such data doesn't need to be rolled over into newer indexes periodically (hourly, daily, or monthly). Keeping this in mind, let's look at a different type of data: time-series data.

As the name indicates, time-series data is time-sensitive and time-dependent. Take the example of logs generated by an Apache web server, shown in figure 3.6.